DSA TREES — PROBLEM SET 1

60 complete coding problems • 20 Easy • 20 Medium • 20 Hard • Java solutions

Format: Problem → Example → Approach → Algorithm → Java Code → Complexity

Easy Problems — 20

1. Maximum Depth of Binary Tree

Problem: Given the root of a binary tree, return its maximum depth. The depth is the number of nodes on the longest root-to-leaf path.

Example Input: root = [3,9,20,null,null,15,7]

Example Output: 3

Approach: Use recursion. The depth of a node is 1 plus the larger depth of its two children.

Algorithm: If root is null return 0; recursively find left and right depths; return 1 + max(left,right).

Java Answer:

```
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h) recursion stack

2. Same Tree

Problem: Given two binary trees, determine whether they are structurally identical and contain the same values.

Example Input: p = [1,2,3], q = [1,2,3]

Example Output: true

Approach: Compare corresponding nodes recursively. Both null means equal; one null or different values means false.

Algorithm: Check root values, then compare left subtrees and right subtrees.

Java Answer:

```
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q;
        return p.val == q.val
            && isSameTree(p.left, q.left)
            && isSameTree(p.right, q.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

3. Invert Binary Tree

Problem: Invert a binary tree by swapping every node's left and right children.

Example Input: root = [4,2,7,1,3,6,9]

Example Output: [4,7,2,9,6,3,1]

Approach: At each node, swap its children and recursively invert both subtrees.

Algorithm: If root is null return null; swap left/right; recurse on both children.

Java Answer:

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode temp = root.left;
        root.left = root.right;
        root.right = temp;
        invertTree(root.left);
        invertTree(root.right);
        return root;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

4. Binary Tree Preorder Traversal

Problem: Return the preorder traversal of a binary tree: Root, Left, Right.

Example Input: root = [1,null,2,3]

Example Output: [1,2,3]

Approach: Use DFS recursion. Visit the current node before its children.

Algorithm: Add root value, recursively traverse left, then right.

Java Answer:

```
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        dfs(root, ans);
        return ans;
    }

    private void dfs(TreeNode node, List<Integer> ans) {
        if (node == null) return;
        ans.add(node.val);
        dfs(node.left, ans);
        dfs(node.right, ans);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

5. Binary Tree Inorder Traversal

Problem: Return the inorder traversal of a binary tree: Left, Root, Right.

Example Input: root = [1,null,2,3]

Example Output: [1,3,2]

Approach: Use recursive DFS and add a node after its left subtree but before its right subtree.

Algorithm: Traverse left; add current value; traverse right.

Java Answer:

```
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        dfs(root, ans);
        return ans;
    }

    private void dfs(TreeNode node, List<Integer> ans) {
        if (node == null) return;
        dfs(node.left, ans);
        ans.add(node.val);
        dfs(node.right, ans);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

6. Binary Tree Postorder Traversal

Problem: Return the postorder traversal: Left, Right, Root.

Example Input: root = [1,null,2,3]

Example Output: [3,2,1]

Approach: Process both children before adding the current node.

Algorithm: Recursively visit left, right, then append current value.

Java Answer:

```
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        dfs(root, ans);
        return ans;
    }

    private void dfs(TreeNode node, List<Integer> ans) {
        if (node == null) return;
        dfs(node.left, ans);
        dfs(node.right, ans);
        ans.add(node.val);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

7. Level Order Traversal

Problem: Return the values of a binary tree level by level from top to bottom.

Example Input: root = [3,9,20,null,null,15,7]

Example Output: [[3],[9,20],[15,7]]

Approach: Breadth-first search naturally processes one tree level at a time.

Algorithm: Put root in a queue. For each level, process queue.size() nodes and enqueue their children.

Java Answer:

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        if (root == null) return ans;

        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);

        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>();
            while (size-- > 0) {
                TreeNode cur = q.poll();
                level.add(cur.val);
                if (cur.left != null) q.offer(cur.left);
                if (cur.right != null) q.offer(cur.right);
            }
            ans.add(level);
        }
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

8. Count Good Nodes in Binary Tree

Problem: A node is good if no node on the path from the root to that node has a value greater than it. Count all good nodes.

Example Input: root = [3,1,4,3,null,1,5]

Example Output: 4

Approach: Carry the maximum value seen on the current root-to-node path.

Algorithm: At each node, check node.val >= maxSeen; update maxSeen and recurse.

Java Answer:

```
class Solution {
    public int goodNodes(TreeNode root) {
        return dfs(root, Integer.MIN_VALUE);
    }

    private int dfs(TreeNode node, int maxSeen) {
        if (node == null) return 0;
        int good = node.val >= maxSeen ? 1 : 0;
        int nextMax = Math.max(maxSeen, node.val);
        return good + dfs(node.left, nextMax)
                    + dfs(node.right, nextMax);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

9. Search in a Binary Search Tree

Problem: Given a BST and a target value, return the node containing the target or null if it does not exist.

Example Input: root = [4,2,7,1,3], val = 2

Example Output: [2,1,3]

Approach: BST ordering lets us discard half of the remaining search path at each step.

Algorithm: If target equals current return current; smaller goes left, larger goes right.

Java Answer:

```
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        if (root == null || root.val == val) return root;
        if (val < root.val) return searchBST(root.left, val);
        return searchBST(root.right, val);
    }
}
```

Time Complexity: O(h) **Space Complexity:** O(h) recursive

10. Insert into a Binary Search Tree

Problem: Insert a value into a BST and return the root of the resulting tree.

Example Input: root = [4,2,7,1,3], val = 5

Example Output: [4,2,7,1,3,5]

Approach: Follow BST ordering until an empty child position is found.

Algorithm: If root is null create a node; otherwise recurse left for smaller and right for larger values.

Java Answer:

```
class Solution {
    public TreeNode insertIntoBST(TreeNode root, int val) {
        if (root == null) return new TreeNode(val);
        if (val < root.val)
            root.left = insertIntoBST(root.left, val);
        else
            root.right = insertIntoBST(root.right, val);
        return root;
    }
}
```

Time Complexity: O(h) **Space Complexity:** O(h)

11. Minimum Depth of Binary Tree

Problem: Return the number of nodes in the shortest root-to-leaf path.

Example Input: root = [3,9,20,null,null,15,7]

Example Output: 2

Approach: A missing child cannot be treated as depth zero when finding a leaf path. Handle one-child nodes separately.

Algorithm: If leaf return 1; if one child is null, use the other depth; otherwise take min(left,right)+1.

Java Answer:

```
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null) return 0;
        if (root.left == null) return 1 + minDepth(root.right);
        if (root.right == null) return 1 + minDepth(root.left);
        return 1 + Math.min(minDepth(root.left), minDepth(root.right));
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

12. Sum of Left Leaves

Problem: Return the sum of all leaf nodes that are left children of their parents.

Example Input: root = [3,9,20,null,null,15,7]

Example Output: 24

Approach: Carry whether the current node is a left child and count it only when it is also a leaf.

Algorithm: At each node, if left-child flag and both children null, add value; otherwise recurse.

Java Answer:

```
class Solution {
    public int sumOfLeftLeaves(TreeNode root) {
        return dfs(root, false);
    }

    private int dfs(TreeNode node, boolean isLeft) {
        if (node == null) return 0;
        if (node.left == null && node.right == null)
            return isLeft ? node.val : 0;
        return dfs(node.left, true) + dfs(node.right, false);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

13. Path Sum

Problem: Determine whether the tree has a root-to-leaf path whose node values add up to targetSum.

Example Input: root = [5,4,8,11,null,13,4,7,2], targetSum = 22

Example Output: true

Approach: Subtract each visited node value from the remaining target. At a leaf, check whether the remainder equals the leaf value.

Algorithm: If null return false; at leaf compare target; otherwise recurse with target-node.val.

Java Answer:

```
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        if (root == null) return false;
        if (root.left == null && root.right == null)
            return root.val == targetSum;
        int rem = targetSum - root.val;
        return hasPathSum(root.left, rem)
            || hasPathSum(root.right, rem);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

14. Symmetric Tree

Problem: Check whether a binary tree is a mirror of itself around its center.

Example Input: root = [1,2,2,3,4,4,3]

Example Output: true

Approach: Compare two subtrees as mirror images: left of one with right of the other and vice versa.

Algorithm: Use a helper taking two nodes; compare values and cross children.

Java Answer:

```
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return root == null || mirror(root.left, root.right);
    }

    private boolean mirror(TreeNode a, TreeNode b) {
        if (a == null || b == null) return a == b;
        return a.val == b.val
            && mirror(a.left, b.right)
            && mirror(a.right, b.left);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

15. Balanced Binary Tree

Problem: Determine whether the height difference between the left and right subtrees of every node is at most one.

Example Input: root = [3,9,20,null,null,15,7]

Example Output: true

Approach: Return subtree height when balanced, otherwise return -1 as a failure marker.

Algorithm: Compute left height and right height. If either is -1 or their difference exceeds one, return -1.

Java Answer:

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return height(root) != -1;
    }

    private int height(TreeNode node) {
        if (node == null) return 0;
        int l = height(node.left);
        if (l == -1) return -1;
        int r = height(node.right);
        if (r == -1 || Math.abs(l - r) > 1) return -1;
        return 1 + Math.max(l, r);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

16. Lowest Common Ancestor of a BST

Problem: Given two nodes in a BST, find their lowest common ancestor.

Example Input: root = [6,2,8,0,4,7,9], p=2, q=8

Example Output: 6

Approach: BST ordering tells whether both targets lie left, both lie right, or split at the current node.

Algorithm: Move left if both values are smaller; right if both are larger; otherwise current node is the LCA.

Java Answer:

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root,
                                         TreeNode p, TreeNode q) {
        while (root != null) {
            if (p.val < root.val && q.val < root.val)
                root = root.left;
            else if (p.val > root.val && q.val > root.val)
                root = root.right;
            else
                return root;
        }
        return null;
    }
}
```

Time Complexity: O(h) **Space Complexity:** O(1)

17. Validate Binary Search Tree

Problem: Determine whether a binary tree satisfies strict BST ordering: every left value is smaller and every right value is larger than all allowed ancestors.

Example Input: root = [2,1,3]

Example Output: true

Approach: Use lower and upper bounds to carry the valid range for each node.

Algorithm: A node must lie strictly between low and high; recurse with updated bounds.

Java Answer:

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean valid(TreeNode node, long low, long high) {
        if (node == null) return true;
        if (node.val <= low || node.val >= high) return false;
        return valid(node.left, low, node.val)
            && valid(node.right, node.val, high);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

18. Kth Smallest Element in a BST

Problem: Return the kth smallest value in a BST.

Example Input: root = [3,1,4,null,2], k = 1

Example Output: 1

Approach: Inorder traversal of a BST visits values in sorted order. Stop after visiting k nodes.

Algorithm: Iteratively perform inorder traversal with a stack and decrement k on each visit.

Java Answer:

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Stack<TreeNode> st = new Stack<>();
        TreeNode cur = root;

        while (true) {
            while (cur != null) {
                st.push(cur);
                cur = cur.left;
            }
            cur = st.pop();
            if (--k == 0) return cur.val;
            cur = cur.right;
        }
    }
}
```

Time Complexity: O(h + k) **Space Complexity:** O(h)

Medium Problems — 20

1. Binary Tree Right Side View

Problem: Return the node values visible when the binary tree is viewed from the right side.

Example Input: root = [1,2,3,null,5,null,4]

Example Output: [1,3,4]

Approach: BFS processes one level at a time. The last node removed from each level is the visible node.

Algorithm: For each level, process exactly its current queue size and record the last node.

Java Answer:

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        if (root == null) return ans;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);

        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                TreeNode cur = q.poll();
                if (i == size - 1) ans.add(cur.val);
                if (cur.left != null) q.offer(cur.left);
                if (cur.right != null) q.offer(cur.right);
            }
        }
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

2. Binary Tree Left Side View

Problem: Return the node values visible when the tree is viewed from the left side.

Example Input: root = [1,2,3,null,5,null,4]

Example Output: [1,2,5]

Approach: The first node encountered at every BFS level is visible from the left.

Algorithm: Record the first node processed for each level.

Java Answer:

```
class Solution {
    public List<Integer> leftSideView(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        if (root == null) return ans;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);

        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                TreeNode cur = q.poll();
                if (i == 0) ans.add(cur.val);
                if (cur.left != null) q.offer(cur.left);
                if (cur.right != null) q.offer(cur.right);
            }
        }
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

3. Zigzag Level Order Traversal

Problem: Traverse the tree level by level, alternating left-to-right and right-to-left.

Example Input: root = [3,9,20,null,null,15,7]

Example Output: [[3],[20,9],[15,7]]

Approach: Use BFS and reverse the insertion order on every second level.

Algorithm: Collect a level into a list; reverse it when direction is right-to-left.

Java Answer:

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        if (root == null) return ans;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        boolean leftToRight = true;

        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>();
            while (size-- > 0) {
                TreeNode cur = q.poll();
                level.add(cur.val);
                if (cur.left != null) q.offer(cur.left);
                if (cur.right != null) q.offer(cur.right);
            }
            if (!leftToRight) Collections.reverse(level);
            ans.add(level);
            leftToRight = !leftToRight;
        }
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

4. Diameter of Binary Tree

Problem: Find the number of edges in the longest path between any two nodes.

Example Input: root = [1,2,3,4,5]

Example Output: 3

Approach: For each node, a path passing through it has left height + right height edges. Track the maximum while computing heights.

Algorithm: Postorder DFS returns height and updates a global diameter with leftHeight + rightHeight.

Java Answer:

```
class Solution {
    private int diameter = 0;

    public int diameterOfBinaryTree(TreeNode root) {
        height(root);
        return diameter;
    }

    private int height(TreeNode node) {
        if (node == null) return 0;
        int l = height(node.left);
        int r = height(node.right);
        diameter = Math.max(diameter, l + r);
        return 1 + Math.max(l, r);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

5. Path Sum II

Problem: Return every root-to-leaf path whose values sum to targetSum.

Example Input: root = [5,4,8,11,null,13,4,7,2], target=22

Example Output: [[5,4,11,2]]

Approach: Use DFS with backtracking. Maintain the current path and remaining sum.

Algorithm: Add node to path; at a leaf check the remaining sum; recurse; remove node before returning.

Java Answer:

```
class Solution {
    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> ans = new ArrayList<>();
        dfs(root, targetSum, new ArrayList<>(), ans);
        return ans;
    }

    private void dfs(TreeNode node, int sum, List<Integer> path,
                     List<List<Integer>> ans) {
        if (node == null) return;
        path.add(node.val);
        sum -= node.val;

        if (node.left == null && node.right == null && sum == 0)
            ans.add(new ArrayList<>(path));

        dfs(node.left, sum, path, ans);
        dfs(node.right, sum, path, ans);
        path.remove(path.size() - 1);
    }
}
```

Time Complexity: O(n * h) worst case **Space Complexity:** O(h) excluding output

6. Lowest Common Ancestor of a Binary Tree

Problem: Find the lowest node that has both p and q in its subtree. The tree is not necessarily a BST.

Example Input: root=[3,5,1,6,2,0,8], p=5, q=1

Example Output: 3

Approach: Unlike a BST, no ordering is available. Recursion can return whether a target was found below each side.

Algorithm: If current is p or q return current. If both recursive sides return non-null, current is LCA; otherwise return the non-null side.

Java Answer:

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root,
                                         TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) return root;

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

        if (left != null && right != null) return root;
        return left != null ? left : right;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

7. Construct Binary Tree from Preorder and Inorder

Problem: Reconstruct a binary tree from preorder and inorder traversals with unique values.

Example Input: preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]

Example Output: [3,9,20,null,null,15,7]

Approach: Preorder gives the root first. Inorder locates the root and splits left and right subtrees.

Algorithm: Store inorder indexes in a map; recursively consume preorder and use inorder bounds.

Java Answer:

```
class Solution {
    private int preIndex = 0;
    private Map<Integer,Integer> pos = new HashMap<>();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        for (int i = 0; i < inorder.length; i++) pos.put(inorder[i], i);
        return build(preorder, 0, inorder.length - 1);
    }

    private TreeNode build(int[] pre, int l, int r) {
        if (l > r) return null;
        int val = pre[preIndex++];
        TreeNode root = new TreeNode(val);
        int m = pos.get(val);
        root.left = build(pre, l, m - 1);
        root.right = build(pre, m + 1, r);
        return root;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

8. Construct Binary Tree from Inorder and Postorder

Problem: Reconstruct a binary tree from inorder and postorder traversals with unique values.

Example Input: inorder=[9,3,15,20,7], postorder=[9,15,7,20,3]

Example Output: [3,9,20,null,null,15,7]

Approach: Postorder's final value is the root. Inorder splits the left and right subtrees.

Algorithm: Consume postorder from right to left; build right subtree before left subtree.

Java Answer:

```
class Solution {
    private int postIndex;
    private Map<Integer,Integer> pos = new HashMap<>();

    public TreeNode buildTree(int[] inorder, int[] postorder) {
        postIndex = postorder.length - 1;
        for (int i = 0; i < inorder.length; i++) pos.put(inorder[i], i);
        return build(postorder, 0, inorder.length - 1);
    }

    private TreeNode build(int[] post, int l, int r) {
        if (l > r) return null;
        int val = post[postIndex--];
        TreeNode root = new TreeNode(val);
        int m = pos.get(val);
        root.right = build(post, m + 1, r);
        root.left = build(post, l, m - 1);
        return root;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

9. Delete Node in a BST

Problem: Delete a key from a BST while preserving BST ordering.

Example Input: root=[5,3,6,2,4,null,7], key=3

Example Output: [5,4,6,2,null,null,7]

Approach: There are three cases: leaf, one child, or two children. For two children, replace the node with its inorder successor.

Algorithm: Search by BST ordering; for two children find minimum of right subtree, copy value, then delete successor.

Java Answer:

```
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;
        if (key < root.val) root.left = deleteNode(root.left, key);
        else if (key > root.val) root.right = deleteNode(root.right, key);
        else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;

            TreeNode s = root.right;
            while (s.left != null) s = s.left;
            root.val = s.val;
            root.right = deleteNode(root.right, s.val);
        }
        return root;
    }
}
```

Time Complexity: O(h) **Space Complexity:** O(h)

10. Kth Largest Element in a BST

Problem: Return the kth largest value in a BST.

Example Input: root=[3,1,4,null,2], k=1

Example Output: 4

Approach: Reverse inorder traversal visits BST values from largest to smallest.

Algorithm: Traverse right-root-left iteratively and stop after k visits.

Java Answer:

```
class Solution {
    public int kthLargest(TreeNode root, int k) {
        Stack<TreeNode> st = new Stack<>();
        TreeNode cur = root;

        while (true) {
            while (cur != null) {
                st.push(cur);
                cur = cur.right;
            }
            cur = st.pop();
            if (--k == 0) return cur.val;
            cur = cur.left;
        }
    }
}
```

Time Complexity: O(h + k) **Space Complexity:** O(h)

11. Binary Tree Maximum Width

Problem: Return the maximum width of a binary tree, counting the positions between the leftmost and rightmost non-null nodes as if the tree were complete.

Example Input: root=[1,3,2,5,3,null,9]

Example Output: 4

Approach: Assign each node a conceptual complete-tree index. Width is lastIndex-firstIndex+1 for a level.

Algorithm: BFS stores node/index pairs. Normalize indexes per level to avoid integer overflow.

Java Answer:

```
class Solution {
    public int widthOfBinaryTree(TreeNode root) {
        if (root == null) return 0;
        Queue<NodePos> q = new LinkedList<>();
        q.offer(new NodePos(root, 0L));
        long ans = 0;

        while (!q.isEmpty()) {
            int size = q.size();
            long base = q.peek().idx;
            long first = 0, last = 0;

            for (int i = 0; i < size; i++) {
                NodePos cur = q.poll();
                long idx = cur.idx - base;
                if (i == 0) first = idx;
                if (i == size - 1) last = idx;
                if (cur.node.left != null)
                    q.offer(new NodePos(cur.node.left, 2 * idx));
                if (cur.node.right != null)
                    q.offer(new NodePos(cur.node.right, 2 * idx + 1));
            }
            ans = Math.max(ans, last - first + 1);
        }
        return (int) ans;
    }

    static class NodePos {
        TreeNode node; long idx;
        NodePos(TreeNode n, long i) { node = n; idx = i; }
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

12. Flatten Binary Tree to Linked List

Problem: Flatten a binary tree in-place into a linked list following preorder traversal. All left pointers must become null.

Example Input: root=[1,2,5,3,4,null,6]

Example Output: [1,null,2,null,3,null,4,null,5,null,6]

Approach: At each node, move its left subtree between the node and its original right subtree.

Algorithm: Find the rightmost node of the left subtree, connect original right there, move left to right, and continue.

Java Answer:

```
class Solution {
    public void flatten(TreeNode root) {
        TreeNode cur = root;
        while (cur != null) {
            if (cur.left != null) {
                TreeNode pred = cur.left;
                while (pred.right != null) pred = pred.right;
                pred.right = cur.right;
                cur.right = cur.left;
                cur.left = null;
            }
            cur = cur.right;
        }
    }
}
```

Time Complexity: O(n) amortized **Space Complexity:** O(1)

13. Sum Root to Leaf Numbers

Problem: Each root-to-leaf path forms a number by concatenating node digits. Return the sum of all such numbers.

Example Input: root=[1,2,3]

Example Output: 25

Approach: Carry the number formed so far. At each node, append its digit using current*10 + node.val.

Algorithm: At a leaf return the formed number; otherwise sum both subtree results.

Java Answer:

```
class Solution {
    public int sumNumbers(TreeNode root) {
        return dfs(root, 0);
    }

    private int dfs(TreeNode node, int cur) {
        if (node == null) return 0;
        int next = cur * 10 + node.val;
        if (node.left == null && node.right == null) return next;
        return dfs(node.left, next) + dfs(node.right, next);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

14. Find Duplicate Subtrees

Problem: Return roots of all subtrees that occur more than once with identical structure and values.

Example Input: root=[1,2,3,4,null,2,4,null,null,4]

Example Output: duplicate subtree roots: [2,4]

Approach: Give every subtree a unique serialization ID. Repeated IDs identify duplicate subtrees.

Algorithm: Postorder serialize as leftID,rightID,value; count each serialization and add a node when count becomes 2.

Java Answer:

```
class Solution {
    Map<String,Integer> ids = new HashMap<>();
    Map<Integer,Integer> freq = new HashMap<>();
    List<TreeNode> ans = new ArrayList<>();
    int nextId = 1;

    public List<TreeNode> findDuplicateSubtrees(TreeNode root) {
        encode(root);
        return ans;
    }

    private int encode(TreeNode node) {
        if (node == null) return 0;
        int l = encode(node.left);
        int r = encode(node.right);
        String key = l + "," + r + "," + node.val;
        int id = ids.computeIfAbsent(key, k -> nextId++);
        int count = freq.getOrDefault(id, 0) + 1;
        freq.put(id, count);
        if (count == 2) ans.add(node);
        return id;
    }
}
```

Time Complexity: O(n) average **Space Complexity:** O(n)

15. Convert Sorted Array to Balanced BST

Problem: Given a sorted array, construct a height-balanced BST.

Example Input: nums=[-10,-3,0,5,9]

Example Output: [0,-3,9,-10,null,5]

Approach: Choosing the middle element as root keeps the two sides nearly equal.

Algorithm: Recursively choose midpoint; left half becomes left subtree and right half becomes right subtree.

Java Answer:

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return build(nums, 0, nums.length - 1);
    }

    private TreeNode build(int[] a, int l, int r) {
        if (l > r) return null;
        int m = l + (r - l) / 2;
        TreeNode root = new TreeNode(a[m]);
        root.left = build(a, l, m - 1);
        root.right = build(a, m + 1, r);
        return root;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(log n)

16. Range Sum of BST

Problem: Return the sum of all BST node values within [low, high].

Example Input: root=[10,5,15,3,7,null,18], low=7, high=15

Example Output: 32

Approach: Use BST ordering to prune branches that cannot contain valid values.

Algorithm: If value is below low, only right can help; if above high, only left can help; otherwise explore both.

Java Answer:

```
class Solution {
    public int rangeSumBST(TreeNode root, int low, int high) {
        if (root == null) return 0;
        if (root.val < low) return rangeSumBST(root.right, low, high);
        if (root.val > high) return rangeSumBST(root.left, low, high);
        return root.val
             + rangeSumBST(root.left, low, high)
             + rangeSumBST(root.right, low, high);
    }
}
```

Time Complexity: O(h + k) typical, O(n) worst **Space Complexity:** O(h)

Hard Problems — 20

1. Binary Tree Maximum Path Sum

Problem: A path may start and end at any nodes but cannot revisit a node. Return the maximum possible sum of node values on one path.

Example Input: root=[-10,9,20,null,null,15,7]

Example Output: 42

Approach: At each node, the best path that continues upward can use only one child. But the global answer may combine both child gains through the node.

Algorithm: Compute max downward gain; clamp negative child gains to zero; update answer with leftGain+node+rightGain.

Java Answer:

```
class Solution {
    private int ans = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        gain(root);
        return ans;
    }

    private int gain(TreeNode node) {
        if (node == null) return 0;
        int left = Math.max(0, gain(node.left));
        int right = Math.max(0, gain(node.right));
        ans = Math.max(ans, node.val + left + right);
        return node.val + Math.max(left, right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

2. Serialize and Deserialize Binary Tree

Problem: Design methods to convert a binary tree to a string and reconstruct the exact same tree.

Example Input: root=[1,2,3,null,null,4,5]

Example Output: same tree after round trip

Approach: Null markers are required because traversal values alone do not preserve structure.

Algorithm: Serialize preorder with a sentinel for nulls; deserialize by consuming tokens recursively.

Java Answer:

```
public class Codec {
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        build(root, sb);
        return sb.toString();
    }

    private void build(TreeNode n, StringBuilder sb) {
        if (n == null) {
            sb.append("#,");
            return;
        }
        sb.append(n.val).append(",");
        build(n.left, sb);
        build(n.right, sb);
    }

    public TreeNode deserialize(String data) {
        String[] a = data.split(",");
        int[] idx = {0};
        return read(a, idx);
    }

    private TreeNode read(String[] a, int[] idx) {
        if (a[idx[0]].equals("#")) {
            idx[0]++;
            return null;
        }
        TreeNode n = new TreeNode(Integer.parseInt(a[idx[0]++]));
        n.left = read(a, idx);
        n.right = read(a, idx);
        return n;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

3. Binary Tree Cameras

Problem: Place the minimum number of cameras so every node is monitored by itself, its parent, or one of its children.

Example Input: root=[0,0,null,0,0]

Example Output: 1

Approach: Use three states: 0 = needs camera, 1 = has camera, 2 = covered. A parent installs a camera when a child needs one.

Algorithm: Postorder: if child needs camera, current gets camera; if child has camera, current is covered; otherwise current needs camera.

Java Answer:

```
class Solution {
    private int cameras = 0;

    public int minCameraCover(TreeNode root) {
        if (dfs(root) == 0) cameras++;
        return cameras;
    }

    private int dfs(TreeNode node) {
        if (node == null) return 2;

        int l = dfs(node.left);
        int r = dfs(node.right);

        if (l == 0 || r == 0) {
            cameras++;
            return 1;
        }
        if (l == 1 || r == 1) return 2;
        return 0;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

4. All Nodes Distance K in Binary Tree

Problem: Given a target node and integer k, return all nodes exactly k edges away from the target.

Example Input: root=[3,5,1,6,2,0,8,null,null,7,4], target=5, k=2

Example Output: [7,4,1]

Approach: Convert the tree into an undirected graph by recording parent pointers, then BFS from target.

Algorithm: First map each node to its parent. BFS can then move left, right, and parent while tracking distance.

Java Answer:

```
class Solution {
    public List<Integer> distanceK(TreeNode root, TreeNode target, int k) {
        Map<TreeNode, TreeNode> parent = new HashMap<>();
        buildParent(root, null, parent);

        Queue<TreeNode> q = new LinkedList<>();
        Set<TreeNode> seen = new HashSet<>();
        q.offer(target);
        seen.add(target);
        int dist = 0;

        while (!q.isEmpty() && dist < k) {
            int size = q.size();
            while (size-- > 0) {
                TreeNode cur = q.poll();
                TreeNode[] next = {cur.left, cur.right, parent.get(cur)};
                for (TreeNode x : next) {
                    if (x != null && seen.add(x)) q.offer(x);
                }
            }
            dist++;
        }

        List<Integer> ans = new ArrayList<>();
        while (!q.isEmpty()) ans.add(q.poll().val);
        return ans;
    }

    private void buildParent(TreeNode n, TreeNode p,
                             Map<TreeNode,TreeNode> map) {
        if (n == null) return;
        map.put(n, p);
        buildParent(n.left, n, map);
        buildParent(n.right, n, map);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

5. Maximum Sum BST in Binary Tree

Problem: Find the maximum sum of values among all subtrees that are valid BSTs.

Example Input: root=[1,4,3,2,4,2,5,null,null,null,null,null,null,4,6]

Example Output: 20

Approach: For each subtree return whether it is a BST plus its minimum, maximum, and sum. A parent is a BST only if both children are BSTs and its value lies between their ranges.

Algorithm: Postorder DP returns {isBST,min,max,sum}. Update global maximum when the current subtree is valid.

Java Answer:

```
class Solution {
    private int ans = 0;

    public int maxSumBST(TreeNode root) {
        dfs(root);
        return ans;
    }

    // {isBST, min, max, sum}
    private int[] dfs(TreeNode n) {
        if (n == null)
            return new int[]{1, Integer.MAX_VALUE, Integer.MIN_VALUE, 0};

        int[] L = dfs(n.left), R = dfs(n.right);

        if (L[0] == 1 && R[0] == 1 &&
            n.val > L[2] && n.val < R[1]) {
            int sum = L[3] + R[3] + n.val;
            ans = Math.max(ans, sum);
            return new int[]{
                1,
                Math.min(n.val, L[1]),
                Math.max(n.val, R[2]),
                sum
            };
        }
        return new int[]{0, 0, 0, 0};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

6. Recover a Swapped BST

Problem: Exactly two node values in a BST have been swapped. Restore the BST without changing its structure.

Example Input: inorder = [1,5,3,4,2,6]

Example Output: inorder becomes [1,2,3,4,5,6]

Approach: A correct BST has sorted inorder traversal. Swapped values create one or two inversions.

Algorithm: Perform inorder traversal and track the previous node. First inversion gives the first bad node; every later inversion updates the second bad node. Swap values.

Java Answer:

```
class Solution {
    TreeNode first, second, prev;

    public void recoverTree(TreeNode root) {
        inorder(root);
        int t = first.val;
        first.val = second.val;
        second.val = t;
    }

    private void inorder(TreeNode n) {
        if (n == null) return;
        inorder(n.left);

        if (prev != null && prev.val > n.val) {
            if (first == null) first = prev;
            second = n;
        }
        prev = n;
        inorder(n.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

7. Burning Binary Tree

Problem: Given a target node, fire starts there at time 0 and spreads to its children and parent each minute. Return the time to burn the whole tree.

Example Input: root=[1,2,3,4,5,6,7], target=5

Example Output: 3

Approach: Treat parent-child links as undirected edges. BFS from the target gives the minimum time to reach every node.

Algorithm: Build parent pointers, then perform BFS and count levels until all reachable nodes are burned.

Java Answer:

```
class Solution {
    public int burnTime(TreeNode root, int target) {
        Map<TreeNode, TreeNode> parent = new HashMap<>();
        TreeNode start = build(root, null, target, parent);

        Queue<TreeNode> q = new LinkedList<>();
        Set<TreeNode> seen = new HashSet<>();
        q.offer(start);
        seen.add(start);
        int time = -1;

        while (!q.isEmpty()) {
            int size = q.size();
            time++;
            while (size-- > 0) {
                TreeNode cur = q.poll();
                TreeNode[] next = {cur.left, cur.right, parent.get(cur)};
                for (TreeNode x : next)
                    if (x != null && seen.add(x)) q.offer(x);
            }
        }
        return time;
    }

    private TreeNode build(TreeNode n, TreeNode p, int target,
                           Map<TreeNode,TreeNode> parent) {
        if (n == null) return null;
        parent.put(n, p);
        if (n.val == target) return n;
        TreeNode x = build(n.left, n, target, parent);
        return x != null ? x : build(n.right, n, target, parent);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

8. Longest ZigZag Path in a Binary Tree

Problem: A ZigZag path alternates left and right child moves. Return its maximum number of edges.

Example Input: root=[1,null,1,1,1,null,null,1,1,null,1]

Example Output: 3

Approach: For each node, maintain the longest path arriving from the left direction and from the right direction.

Algorithm: If the previous move was left, the next must go right, so swap the state when descending.

Java Answer:

```
class Solution {
    private int ans = 0;

    public int longestZigZag(TreeNode root) {
        dfs(root, 0, 0);
        return ans;
    }

    // leftLen = path ending with a left move, rightLen with right move
    private void dfs(TreeNode n, int leftLen, int rightLen) {
        if (n == null) return;
        ans = Math.max(ans, Math.max(leftLen, rightLen));
        dfs(n.left, rightLen + 1, 0);
        dfs(n.right, 0, leftLen + 1);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

9. Largest BST Subtree

Problem: Find the number of nodes in the largest subtree that is itself a valid BST.

Example Input: root=[10,5,15,1,8,null,7]

Example Output: 3

Approach: Postorder DP can determine whether each subtree is a BST and its size, min, and max.

Algorithm: A subtree is valid if both children are valid and leftMax < node < rightMin. Track the largest valid size.

Java Answer:

```
class Solution {
    private int ans = 0;

    public int largestBSTSubtree(TreeNode root) {
        dfs(root);
        return ans;
    }

    // {valid, min, max, size}
    private int[] dfs(TreeNode n) {
        if (n == null)
            return new int[]{1, Integer.MAX_VALUE, Integer.MIN_VALUE, 0};

        int[] L = dfs(n.left), R = dfs(n.right);

        if (L[0] == 1 && R[0] == 1 &&
            n.val > L[2] && n.val < R[1]) {
            int size = L[3] + R[3] + 1;
            ans = Math.max(ans, size);
            return new int[]{
                1, Math.min(n.val, L[1]),
                Math.max(n.val, R[2]), size
            };
        }
        return new int[]{0, 0, 0, 0};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

10. Count Complete Tree Nodes in O(log² n)

Problem: Count nodes in a complete binary tree faster than a full traversal.

Example Input: root=[1,2,3,4,5,6]

Example Output: 6

Approach: In a complete tree, if leftmost and rightmost heights are equal, the subtree is perfect and its node count is known directly.

Algorithm: Compute left and right heights. If equal return 2^h-1; otherwise recursively count both children plus one.

Java Answer:

```
class Solution {
    public int countNodes(TreeNode root) {
        if (root == null) return 0;
        int lh = leftHeight(root);
        int rh = rightHeight(root);
        if (lh == rh) return (1 << lh) - 1;
        return 1 + countNodes(root.left) + countNodes(root.right);
    }

    private int leftHeight(TreeNode n) {
        int h = 0;
        while (n != null) { h++; n = n.left; }
        return h;
    }

    private int rightHeight(TreeNode n) {
        int h = 0;
        while (n != null) { h++; n = n.right; }
        return h;
    }
}
```

Time Complexity: O(log² n) **Space Complexity:** O(log n) recursion

11. Distance Between Two Nodes

Problem: Given two node values in a binary tree, return the number of edges between them.

Example Input: root=[1,2,3,4,5,6,7], a=4, b=7

Example Output: 4

Approach: Distance(a,b) = depth(a)+depth(b)-2*depth(LCA). Find the LCA and then depths.

Algorithm: Find LCA recursively, then compute distance from LCA to each target.

Java Answer:

```
class Solution {
    public int distance(TreeNode root, int a, int b) {
        TreeNode lca = lca(root, a, b);
        return depth(lca, a) + depth(lca, b);
    }

    private TreeNode lca(TreeNode n, int a, int b) {
        if (n == null || n.val == a || n.val == b) return n;
        TreeNode l = lca(n.left, a, b);
        TreeNode r = lca(n.right, a, b);
        if (l != null && r != null) return n;
        return l != null ? l : r;
    }

    private int depth(TreeNode n, int x) {
        if (n == null) return -1;
        if (n.val == x) return 0;
        int l = depth(n.left, x);
        if (l != -1) return l + 1;
        int r = depth(n.right, x);
        return r == -1 ? -1 : r + 1;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

12. Minimum Vertex Cover on a Tree

Problem: Choose the minimum number of tree nodes so every edge has at least one selected endpoint. The tree is rooted at any node.

Example Input: Edges: 1-2, 1-3, 3-4, 3-5

Example Output: 2

Approach: Tree DP has two states: take the current node or skip it. If skipped, every child must be taken.

Algorithm: take = 1 + min(takeChild, skipChild); skip = sum(takeChild). Return min(root states).

Java Answer:

```
class Solution {
    public int minVertexCover(TreeNode root) {
        int[] ans = dfs(root);
        return Math.min(ans[0], ans[1]);
    }

    // [take, skip]
    private int[] dfs(TreeNode n) {
        if (n == null) return new int[]{0, 0};

        int[] L = dfs(n.left);
        int[] R = dfs(n.right);

        int take = 1 + Math.min(L[0], L[1])
                     + Math.min(R[0], R[1]);
        int skip = L[0] + R[0];

        return new int[]{take, skip};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

13. Tree Maximum Independent Set

Problem: Select a maximum-sum set of tree nodes such that no parent and child are both selected.

Example Input: root values=[3,2,3,null,3,null,1]

Example Output: 7

Approach: This is tree DP. For each node calculate the best sum if it is selected and if it is skipped.

Algorithm: take = value + skip(left)+skip(right); skip = max(take,skip) for both children.

Java Answer:

```
class Solution {
    public int maxIndependentSet(TreeNode root) {
        int[] dp = dfs(root);
        return Math.max(dp[0], dp[1]);
    }

    // [take, skip]
    private int[] dfs(TreeNode n) {
        if (n == null) return new int[]{0, 0};
        int[] L = dfs(n.left), R = dfs(n.right);

        int take = n.val + L[1] + R[1];
        int skip = Math.max(L[0], L[1])
                 + Math.max(R[0], R[1]);

        return new int[]{take, skip};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

14. Sum of Distances in Tree

Problem: For an undirected tree with n nodes, return for every node the sum of distances to all other nodes.

Example Input: n=6, edges=[[0,1],[0,2],[2,3],[2,4],[2,5]]

Example Output: [8,12,6,10,10,10]

Approach: Use rerooting DP. First compute subtree sizes and the distance sum from an arbitrary root. Then move the root across each edge.

Algorithm: For edge parent->child: ans[child] = ans[parent] - size[child] + (n-size[child]).

Java Answer:

```
class Solution {
    List<Integer>[] g;
    int[] size, ans;
    int n;

    public int[] sumOfDistancesInTree(int n, int[][] edges) {
        this.n = n;
        g = new ArrayList[n];
        for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
        for (int[] e : edges) {
            g[e[0]].add(e[1]);
            g[e[1]].add(e[0]);
        }

        size = new int[n];
        ans = new int[n];
        post(0, -1);
        reroot(0, -1);
        return ans;
    }

    private void post(int u, int p) {
        size[u] = 1;
        for (int v : g[u]) if (v != p) {
            post(v, u);
            size[u] += size[v];
            ans[0] += size[v];
        }
    }

    private void reroot(int u, int p) {
        for (int v : g[u]) if (v != p) {
            ans[v] = ans[u] - size[v] + (n - size[v]);
            reroot(v, u);
        }
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

15. Binary Lifting for K-th Ancestor

Problem: Preprocess a rooted tree so that queries asking for the kth ancestor of a node can be answered quickly.

Example Input: parent=[-1,0,0,1,1], query=(4,2)

Example Output: 0

Approach: Store up[node][j] = the 2^j-th ancestor. Any k can be decomposed into powers of two.

Algorithm: Build the ancestor table with dynamic programming. For each set bit of k, jump using up[node][j].

Java Answer:

```
class TreeAncestor {
    private int[][] up;
    private int LOG;

    public TreeAncestor(int n, int[] parent) {
        LOG = 1;
        while ((1 << LOG) <= n) LOG++;
        up = new int[n][LOG];

        for (int i = 0; i < n; i++) up[i][0] = parent[i];
        for (int j = 1; j < LOG; j++) {
            for (int i = 0; i < n; i++) {
                int p = up[i][j - 1];
                up[i][j] = (p == -1) ? -1 : up[p][j - 1];
            }
        }
    }

    public int getKthAncestor(int node, int k) {
        for (int j = 0; j < LOG && node != -1; j++) {
            if ((k & (1 << j)) != 0)
                node = up[node][j];
        }
        return node;
    }
}
```

Time Complexity: Preprocess O(n log n), query O(log n) **Space Complexity:** O(n log n)